

Sprawozdanie 5

Kacper Dusza 131427

Jako danych użyłem pliku SMSSpamCollection.tsv. Podzielono plik na część treningową i testową, gdzie część testowa to 30%. Utworzyłem pipeline, który łączy etap ekstrakcji cech oraz model klasyfikacji. Zastosowano GridSearchCV z 5-krotną walidacją krzyżową w celu przetestowania kombinacji parametrów: metody oraz zakresu n-gramów.

```
main.py x
1 import pandas as pd
2 from sklearn.model_selection import GridSearchCV, train_test_split
3 from sklearn.pipeline import Pipeline
4 from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
5 from sklearn.linear_model import LogisticRegression
6
7 pd.set_option(*args: 'display.max_columns', None)
8 pd.set_option(*args: 'display.max_rows', None)
9 pd.set_option(*args: 'display.max_colwidth', None)
10 pd.set_option(*args: 'display.width', 1000)
11
12 data = pd.read_csv(filepath_or_buffer: 'SMSSpamCollection.tsv', sep='\t', names=['label', 'text'])
13
14 X_train, X_test, Y_train, Y_test = train_test_split(*arrays: data['text'], data['label'], test_size=0.3, random_state=42)
15
16 pl = Pipeline([
17     ('vect', CountVectorizer()),
18     ('clf', LogisticRegression(max_iter=1000))
19 ])
20
21
22 params = [
23     {
24         'vect' : [CountVectorizer()],
25         'vect__ngram_range' : [(1,1), (1,2)],
26     },
27     {
28         'vect' : [TfidfVectorizer()],
29         'vect__ngram_range' : [(1,1), (1,2)],
30     }
31 ]
```

```

33 grid = GridSearchCV(pl, params, cv=5, n_jobs=-1, verbose=1)
34 grid.fit(X_train, Y_train)
35
36 result = pd.DataFrame(grid.cv_results_)
37 result_disp = result[[
38     'params',
39     'mean_test_score',
40     'std_test_score',
41     'mean_fit_time',
42     'std_fit_time',
43     'rank_test_score'
44 ]]
45
46 result_disp = result_disp.sort_values(by='rank_test_score', ascending=True)
47
48 print(result_disp)

```

Wyniki testu są następujące:

```

/home/debian/Pulpit/UNIVERSITY/.venv/bin/python /home/debian/Pulpit/UNIVERSITY/semester_6/language_processing/ex5/main.py
Fitting 5 folds for each of 4 candidates, totalling 20 fits

```

	params	mean_test_score	std_test_score	mean_fit_time	std_fit_time	rank_test_score
0	{'vect': CountVectorizer(), 'vect__ngram_range': (1, 1)}	0.978955	0.006007	0.044352	0.000833	1
1	{'vect': CountVectorizer(), 'vect__ngram_range': (1, 2)}	0.977929	0.005104	0.115007	0.021972	2
2	{'vect': TfidfVectorizer(), 'vect__ngram_range': (1, 1)}	0.965098	0.008805	0.040774	0.002205	3
3	{'vect': TfidfVectorizer(), 'vect__ngram_range': (1, 2)}	0.953038	0.008590	0.137717	0.008009	4

```

Process finished with exit code 0

```

Dla danych z pliku SMSSpamCollection.tsv najlepszym wyborem jest CountVectorizer przy ustawieniu n-gramów na wartości (1,1). Wariant ten osiągnął najwyższą wartość mean_test_score. Można zauważyć zależność, że różnica w czasie trenowania oby modeli jest bardzo mała. Czas uczenia zwiększa się w przypadku zwiększenia parametru n-gram dla obu technik. CountVectorizer osiągał lepszą stabilność co można zaobserwować przy wyniku std_test_score.